Linux AMP
开发指南

版本号: 1.4

发布日期: 2025.7.4

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.2.7	XAA0250	初始版本。
1.1	2023.2.13	XAA0309	根据已有框架完善全文内容。
1.2	2023.4.18	XAA0309	根据要求修改全文格式。
1.3	2023.5.30	XAA0309	修改源码结构以及模块配置部分。
1.4	2025.7.4	KPA0576	修改源码结构, 更新资源表结构，补充驱动层通信例子。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 概述 1
1.1 编写目的 1
1.2 使用范围 1
1.3 相关人员 1

2 术语及概念 2

3 资源分配 3
3.1 DSP 资源 3
3.2 RV 资源 7

4 软件框架配置 9
4.1 Linux AMP 软件框架 9
4.1.1 源码结构 9
4.1.2 模块配置介绍 9
4.1.2.1 device tree 配置 9
4.1.2.2 board.dts 板级配置 10
4.1.2.3 重点属性解释 11
4.1.3 defconfig 配置 13
4.2 RTOS OpenAMP 软件框架 13
4.2.1 源码结构 13
4.2.2 defconfig 配置 13

5 AMP 软件框架介绍 14
5.1 AMP 各层之间的关联 14
5.1.1 层级结构 14
5.1.2 层级作用 14
5.1.2.1 virtio(虚拟化模块) 14
5.1.2.2 RPMsg(远程处理器消息传递) 15
5.1.2.3 RemoteProc(远程处理器控制) 15
5.2 remoteproc 16
5.2.1 resouce_table 资源表 16
5.2.2 Linux 下 remoteproc 相关 API 19
5.2.2.1 rproc 结构体 19
5.2.2.2 启动远程处理器 22
5.2.2.3 停止远程处理器 23
5.2.3 msgbox 通道的申请与释放 24
5.2.4 Linux 侧 sysfs 和 debugfs 各个节点的作用 24
5.3 RPmsg 26

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.3.1 相关结构体 26
5.3.2 API 函数 28
5.3.2.1 rpmsg_send_offchannel_raw() 28
5.3.2.2 rpmsg_send() 28
5.3.2.3 rpmsg_sendto() 29
5.3.2.4 rpmsg_send_offchannel() 29
5.3.2.5 rpmsg_trysend() 30
5.3.2.6 rpmsg_trysendto() 30
5.3.2.7 rpmsg_trysend_offchannel() 30

6 AMP 基本的使用方法 31
6.1 异构启停 31
6.2 异构通信测试 31
6.2.1 Linux 端测试代码: 32
6.2.2 远端处理器测试代码: 32
6.2.3 驱动层 rpmsg 传输 33

广州星翼电子科技有限公司zdyzlinux

插　图

图 3-1　sp_mem_rsc_table 3
图 3-2　dsp_mem_rsc_table1 4
图 3-3　rv_mem_rsc_table 7
图 5-1　resource_table 结构体 17
图 5-2　fw_rsc_hdr 结构体 17
图 5-3　fw_resource_type 资源条目的类型 18
图 5-4　rtos_resource_table 19

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 概述

1.1 编写目的

介绍 AMP 软件框架，为相关人员提供指导。

1.2 使用范围

适用于 Linux-5.4 Linux-5.15 内核。

1.3 相关人员

AMP 软件框架的开发人员/维护人员等。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 术语及概念

术语	解释说明
AMP	多核异构处理器
主处理器	负责控制其它核心以及整个系统运行的核心处理器
远端处理器	被主处理器控制来完成特定任务的处理器 (如 DSP)

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 资源分配

本章节主要介绍 DSP 和 RV 的内存资源，并且介绍 DSP 和 ARM，RV 和 ARM 的地址映射关系。由于外设的地址在不同的 core 上是一致的，且都是共享的，本章节仅关注内存资源。

3.1 DSP 资源

本小节主要介绍 DSP 能够访问的内存资源，以 T113 为例，可参考 usermanual 的《T113-i_Serial_User_Manual》,3.1 Memory Mapping 章节：

The following is the system memory mapping seen by the DSP0 host.

Module	Cacheable Property	Address	Size	Address Mapping Description
SRAM A1	Non-Cacheable (Total 512 MB)	0x0002 0000---0x0002 7FFF	32 KB	The start address of the default DSP.
DSP0 IRAM		0x0002 8000---0x0003 7FFF	64 KB	The DSP0 accesses the address through the external bus to operate the internal local SRAM.
DSP0 DRAM0		0x0003 8000---0x0003 FFFF	32 KB	
DSP0 DRAM1		0x0004 0000---0x0004 7FFF	32 KB	
DSP0 IRAM		0x0040 0000---0x0040 FFFF	64 KB	The DSP0 accesses the address through the internal bus to directly access the local SRAM, which is more efficient than the external bus.
DSP0 DRAM0		0x0042 0000---0x0042 7FFF	32 KB	
DSP0 DRAM1		0x0044 0000---0x0044 7FFF	32 KB	
L4_CONN_periph		0x0200 0000---0x09FF FFFF	128 MB	System Peripherals

图 3-1: sp_mem_rsc_table

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Module	Cacheable Property	Address	Size	Address Mapping Description
DSP_SYS_CFG		0x0170 0000---0x0170 0FFF	4 KB	Peripherals
DRAM SPACE		0x1000 0000---0x1FFF FFFF	256 MB	The front 256 MB space of DDR (0x40000000–0x4FFFFFFF) It requires DSP output AHB for address remapping
Cacheable Mapping Area				
SRAM	Cacheable (Total 512 MB)	0x2002 0000---0x2002 7FFF	32 KB	SRAM A1 has two addresses.
DSP0 IRAM		0x2002 8000---0x2003 7FFF	64 KB	The DSP0 accesses the address through the external bus to operate the internal local SRAM.
DSP0 DRAM0		0x2003 8000---0x2003 FFFF	32 KB	
DSP0 DRAM1		0x2004 0000---0x2004 7FFF	32 KB	
DRAM SPACE		0x3000 0000---0x3FFF FFFF	256 MB	The front 256 MB space of DDR (0x40000000–0x47FFFFFF) It requires DSP output AHB for address remapping
DRAM Special Mapping Area				
DRAM SPACE	Non-Cacheable	0x4000 0000---0x7FFF FFFF	1 GB	
DRAM SPACE	Cacheable Configurable	0x8000 0000---0xBFFF FFFF	1 GB	Reserved space
DRAM SPACE	Cacheable	0xC000 0000---0xFFFF FFFF	1 GB	The front 1 GB space of DDR (0x40000000–0x7FFFFFFF)

图 3-2: dsp_mem_rsc_table1

Module	cacheable 属性	Address	Size	地址映射说明
SRAM A1	non-cacheable	0x000200000 - 0x00027fff	32KB	片内 sram
DSP IRAM	non-cacheable	0x0002_0000 - 0x0002_FFFF	64KB	DSP 通过外部总线访
DSP DRAM0	non-cacheable	0x0003_0000 - 0x0003_7FFF	32KB	DSP 通过外部总线访问

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Module	cacheable 属性	Address	Size	地址映射说明
DSP DRAM1	non-cacheable	0x0003_8000 - 0x0003_FFFF	32KB	DSP 通外部总线访问
DSP IRAM	non-cacheable	0x0040_0000 - 0x0040_FFFF	64KB	DSP 通过内部总线访问
DSP DRAM0	non-cacheable	0x0042_0000 - 0x0042_7FFF	32KB	DSP 通过内部总线访问
DSP DRAM1	non-cacheable	0x0044_0000 - 0x0044_7FFF	32KB	DSP 通过内部总线访问
DRAM SPACE	non-cacheable	0x1000_0000 - 0x1FFF_FFFF	256M	对应 DDRR 的前 256M 空间 (需要 DSP 输出 ahb 做地址重映射)
DSP IRAM	cacheable	0x2002_0000 - 0x2002_FFFF	64KB	DSP 通过外部总线访问
DSP DRAM0	cacheable	0x2003_0000 - 0x2003_7FFF	32KB	DSP 通过外部总线访问
DSP DRAM1	cacheable	0x2003_8000 - 0x2003_FFFF	32KB	DSP 通过外部总线访问

表 2: 目前平台 dsp 的运行内存外部 dram 地址在 0x3000_0000 - 0x3FFF_FFFF，实际访问外部 dram 内存物理地址是

Module	cacheable 属性	Address	Size	地址映射说明
DRAM SPACE	cacheable	0x3000_0000 - 0x3FFF_FFFF	256M	对应 DDRR 的前 256M 空间 (需要 DSP 输出 ahb 做地址重映射)
DRAM SPACE	non-cacheable	0x4000_0000 - 0x7FFF_FFFF	1G	DRAM 特殊映射
DRAM SPACE	cacheable 可配置	0x8000_0000 - 0xBFFF_FFFF	1G	预留空间
DRAM SPACE	cacheable	0xC000_0000 - 0xFFFF_FFFF	1G	直接映射到 DDR 对应空间

由于 DSP 的 cacheable 空间大小只能按照你 512MB 去划分，而且跳转指令不能超过 1G 空间，因此对于 SRAM、以及 DRAM 的空间，DSP 存在多套地址映射，根据不同的地址段区分 cacheable 和 non-cacheable。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

DSP 的 local sram 包括 IRAM 和 DRAM0/1，放在 DSP 内部，其他主机可以通过 DSP 的总线接口对该 RAM 进行访问，而 DSP 可以通过特定地址（0x0040_0000 - 0x0044_FFFF）对其直接访问，实现最高效率。

Module	cacheable 属性	Address	Size	地址映射说明
DSP IRAM	non-cacheable	0x0002_0000 - 0x0002_FFFF	64 KB	DSP 通过外部总线访问
DSP DRAM0	non-cacheable	0x0003_0000 - 0x0003_7FFF	32 KB	DSP 通过外部总线访问
DSP DRAM1	non-cacheable	0x0003_8000 - 0x0003_FFFF	32 KB	DSP 通外部总线访问
DSP IRAM	non-cacheable	0x0040_0000 - 0x0040_FFFF	64 KB	DSP 通过内部总线访问
DSP DRAM0	non-cacheable	0x0042_0000 - 0x0042_7FFF	32 KB	DSP 通过内部总线访问
DSP DRAM1	non-cacheable	0x0044_0000 - 0x0044_7FFF	32 KB	DSP 通过内部总线访问
DRAM SPACE	non-cacheable	0x1000_0000 - 0x1FFF_FFFF	256 MB	对应 DDRR 的前 256M 空间 (需要 DSP 输出 ahb 做地址重映射)
DSP IRAM	cacheable	0x2002_0000 - 0x2002_FFFF	64 KB	DSP 通过外部总线访问
DSP DRAM0	cacheable	0x2003_0000 - 0x2003_7FFF	32 KB	DSP 通过外部总线访问
DSP DRAM1	cacheable	0x2003_8000 - 0x2003_FFFF	32 KB	DSP 通过外部总线访问
DRAM SPACE	cacheable	0x3000_0000 - 0x3FFF_FFFF	256 MB	对应 DDRR 的前 256M 空间 (需要 DSP 输出 ahb 做地址重映射)
DRAM SPACE	non-cacheable	0x4000_0000 - 0x7FFF_FFFF	1 GB	DRAM 特殊映射
DRAM SPACE	cacheable 可配置	0x8000_0000 - 0xBFFF_FFFF	1 GB	预留空间
DRAM SPACE	cacheable	0xC000_0000 - 0xFFFF_FFFF	1 GB	直接映射到 DDR 对应空间

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2 RV 资源

本小节主要介绍 RV 能够访问的内存资源，以 MR153 为例，可参考 usermanual 的《MR153_User Manual》Memory Map 章节：

The following table shows the memory map viewed from the E907.

Region	Address	Size	Description
SRAM			
SRAMC	0x0004 0000---0x0006 7FFF	160 KB	
DE_SHARE_SRAM	0x0006 8000---0x0006 BFFF	16 KB	
Peripheral Space			
MCU_CFG	0x01A0 0000---0x01A0 0FFF	4 KB	
WDT_MCU	0x01A0 2000---0x01A0 2FFF	4 KB	
MCU_LCNT	0x01A0 4000---0x01A0 4FFF	4 KB	
MSGBOX_MCU	0x01A0 6000---0x01A0 6FFF	4 KB	
TIMER_MCU	0x01A0 8000---0x01A0 8FFF	4 KB	
L4_TOP	0x0190 0000---0x0190 0FFF 0x01B0 0000---0x1FFF FFFF	/	system domain, consistent with CPUX memory mapping
MCU_PAHB	0x2FF0 0000---0x2FFF FFFF	1 MB	
E907 TCIP			
MCU_CLINT	0x3000 0000---0x3000 FFFF	64 KB	
MCU_CLIC	0x3080 0000---0x30804 FFFF	20 KB	
MCU_SYSMAP	0x3FFF F000---0x3FFF FFFF	4 KB	
DRAM Space			
DRAM SPACE	0x4000 0000---0x7FFF FFFF	1 GB	Fixed access to the first 1 GB peripheral space and the first 3 GB DDR space by default.
DRAM SPACE	0x8000 0000---0xBFFF FFFF	1 GB	
DRAM SPACE	0xC000 0000---0xFFFF FFFF	1 GB	
E907 accesses DRAM space through AXI. Address mapping is required.			
DRAM SPACE	0x1 0000 0000---0x1 3FFF FFFF	1 GB	When E907_DDR_REMAP=3'b1, map 0xC000 0000---0xFFFF FFFF to 0x1 0000 0000---0x1 3FFF FFFF.

图 3-3: rv_mem_rsc_table

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Module	cacheable 属性	Address	Size	地址映射说明
SRAM	non-cacheable	0x0004_0000 - 0x0006B_FFF	160KB+ 16KB	sram 地址范围，可作为运行内存。
Peripheral Space	non-cacheable	0x1a00000- 0x3ffffffff	/	外设 io 空间，注意 MCU_CLINT/ MCU_CLIC 为 e907 中断控制器地址，MCU_SYSMAP 为内存属性配置。
DRAM SPACE	cacheable	0x40000000 - 0x7ffffff	设备树中配置预留内存	目前 sdk e907 运行的 dram 空间。

广州星翼电子科技有限公司zdyzlinux

4 软件框架配置

本章节主要介绍 AMP 软件框架的配置。

4.1 Linux AMP 软件框架

4.1.1 源码结构

```
sdk/bsp/drivers/remoteproc
├── platform -- 存放芯片平台异构核驱动文件
├── subdev -- 子设备驱动如看门狗，ecc等
├── sunxi_rproc_boot.c -- 异构核引导流程回调接口
├── sunxi_rproc_boot.h -- 异构核引导流程回调头文件
├── sunxi_rproc.c -- 对接remoteproc内核框架接口
├── sunxi_rproc_common_boot.c -- 通用异构时钟时序接口
├── sunxi_rproc_firmware.c --异构核elf解析接口
├── sunxi_rproc_firmware.h -- 异构核elf解析接口头文件
├── sunxi_rproc_internal.h -- 驱动内部成员定义
├── sunxi_rproc_load_partition.c -- 分区读取固件接口
├── sunxi_rproc_load_partition.h -- 分区读取固件头文件
├── sunxi_rproc_standby.c -- 休眠唤醒相关
├── sunxi_rproc_standby.h -- 休眠唤醒接口
├── sunxi_rproc_subdev.c -- 子设备管理文件
├── sunxi_rproc_subdev.h -- 子设备管理头文件
├── sunxi_rproc_trace.c -- 异构核log trace dum接口
├── sunxi_rproc_trace.h -- 异构核log trace dum头文件
├── sunxi_share_interrupt --gpio共享中断接口
└── trace_event -- 异构核事件通知接口
```

4.1.2 模块配置介绍

在不同的 Sunxi 硬件平台中，memory-mappings 等信息不同；但对于同一块板子上的基本配置类似，本小节以 T527 DSP 为例，Linux 侧 AMP 的配置如下 (若需新增 RV 节点，可参考下方内容进行修改配置)。

4.1.2.1 device tree 配置

在 soc 级的 dtsi 文件中提炼了中断控制、时钟等共性信息，是该类芯片所有平台的模块配置，soc 级的 dtsi 文件的路径为：sdk/bsp/configs/{linux-ver}/{CHIP}.dtsi(CHIP 为研发代号，如 sun50iw10p1 等)，DSP 的设备树配置如下所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
dsp0_rproc: dsp0_rproc@0 {
    compatible = "allwinner,hifi4-rproc";
    clock-frequency = <600000000>;
    clocks = <&ccu CLK_PLL_PERI0_2X>, <&dsp_ccu CLK_DSP_DSP>, <&ccu CLK_DSP>, <&dsp_ccu CLK_BUS_DSP_CFG
      >, <&r_ccu CLK_R_AHB>;
    clock-names = "pll", "dsp-mod", "mod", "cfg", "ahbs";
    resets = <&dsp_ccu RST_BUS_DSP>, <&dsp_ccu RST_BUS_DSP_CFG>, <&dsp_ccu RST_BUS_DSP_DBG>;
    reset-names = "mod-rst", "cfg-rst", "dbg-rst";
    reg = <0x0 0x07010364 0x0 0x04>,
      <0x0 0x07100000 0x0 0x40>;
    reg-names = "sram-for-cpux", "hifi4-cfg";
    power-domains = <&pd A523_PD_DSP>;
    status = "okay";
};
```

4.1.2.2 board.dts 板级配置

board.dts 用于保存每一个板级平台的设备信息（如 demo 板，perf1 板，ver1 板等等），里面的配置信息会覆盖上面的 device tree 默认配置信息。board.dts 的路径为 sdk/device/config/chips/${SoC}/configs/${BOARD}/${KERNEL_VER}/board.dts。

对应 board.dts 里面 DSP 的具体配置如下：

```
reserved-memory {
    dsp0ddr_reserved: dsp0ddr@42000000 {
        reg = <0x0 0x42000000 0x0 0x00100000>;
        no-map;
    };

    /*
     * The name should be "vdev%dbuffer".
     * Its size should be not less than
     *    RPMSG_BUF_SIZE * (num of buffers in a vring) * 2
     *   = 512 * (num of buffers in a vring) * 2
     */
    dsp_vdev0buffer: vdev0buffer@42200000 {
        compatible = "shared-dma-pool";
        reg = <0x0 0x42200000 0x0 0x40000>;
        no-map;
    };

    /*
     * The name should be "vdev%dvring%d".
     * The size of each should be not less than
     *    PAGE_ALIGN(vring_size(num, align))
     *   = PAGE_ALIGN(16 * num + 6 + 2 * num + (pads for align) + 6 + 8 * num)
     *
     * (Please refer to the vring layout in include/uapi/linux/virtio_ring.h)
     */
    dsp_vdev0vring0: vdev0vring0@42240000 {
        reg = <0x0 0x42240000 0x0 0x2000>;
        no-map;
    };
    dsp_vdev0vring1: vdev0vring1@42242000 {
        reg = <0x0 0x42242000 0x0 0x2000>;
        no-map;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    };

    /*
    * dsp ram addr
    */
    dsp0iram_reserved: dsp0iram@400000 {
      reg = <0x0 0x400000 0x0 0x10000>;
      no-map;
    };
    dsp0dram0_reserved: dsp0dram0@420000 {
      reg = <0x0 0x420000 0x0 0x8000>;
      no-map;
    };
    dsp0dram1_reserved: dsp0dram1@440000 {
      reg = <0x0 0x440000 0x0 0x8000>;
      no-map;
    };

    dsp_share_space@42100000 {
      no-map;
      reg = <0x0 0x42100000 0x0 0x10000>;
    };
  };

  dsp0_rproc: dsp0_rproc@0 {
    mboxes = <&msgbox 4>;
    mbox-names = "arm-kick";
    memory-region = <&dsp0ddr_reserved>, <&dsp_vdev0buffer>, <&dsp_vdev0vring0>, <&dsp_vdev0vring1>,
        <&dsp0iram_reserved>, <&dsp0dram0_reserved>, <&dsp0dram1_reserved>;
    memory-mappings =
      /* < DA    len    PA >    */
      /* local SRAM via external bus */
      < 0x20000   0x20000    0x20000 >,
      /* local SRAM via internal bus */
      < 0x400000  0x10000    0x400000 >,
      < 0x420000  0x8000     0x420000 >,
      < 0x440000  0x8000     0x440000 >,
      /* DDR front 256MB */
      < 0x10000000    0x10000000  0x40000000 >,
      /* local SRAM via internal bus */
      < 0x20020000    0x10000    0x400000 >,
      < 0x20030000    0x8000     0x420000 >,
      < 0x20038000    0x8000     0x440000 >,

      /* DDR front 256MB */
      < 0x30000000    0x10000000  0x40000000 >,
      /* DDR front 1GB */
      < 0x40000000    0x40000000  0x40000000 >,
      /* DDR front 1GB */
      < 0x80000000    0x40000000  0x40000000 >,
      /* DDR front 1GB */
      < 0xC0000000    0x40000000  0x40000000 >;
    status = "okay";
  };
```

4.1.2.3 重点属性解释

dsp0_rproc 节点，重点属性解释如下 (e906_rproc 节点属性含义相同)：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 与 IC 相关，无法修改的属性：
  - clock-frequency：dsp 主频；
  - clocks：dsp 启动所需要的时钟，例如：pll，dsp 模块时钟，dsp-config 时钟，r_ahb 时钟。在最小系统完成适配之后，会提供时钟的相关文档，时钟的配置要参考文件中定义的时钟名，一般情况下时钟名是不会发生变化的；
  - resets：dsp 启动所需要的复位时钟，例如：dsp 模块 reset 时钟，dsp-config reset 时钟，dsp-dbg reset 时钟。格式：与时钟格式类似；
  - reg：linux 端需要访问的 dsp 的寄存器：dsp 的配置寄存器和 local memory 的 remap 寄存器。在 64 位系统中使用 0x0 进行占位；
  - memory-region：对应上文所说的预留内存的节点；
  - memory-mappings：DA：远程处理器的总线地址；len：地址空间长度；PA：cpux 的总线地址；
  - id：处理器 id，cpux 为 0。
- 用户可二次修改定制的属性：
  - mboxes：linux 端 mailbox 通道，用来表示向远端处理器的该通道写以及远端处理器从 ARM 的该通道读。（Linux MSGBOX 走的是 mailbox 框架，mailbox controller 需要管控所有 msgbox 通道，在配置 moboxes 的通道数时，CPUS 核的通道数为 0-3；DSP 核的通道数为 4-7，需要减 4 为真实物理通道号；RV 核的通道数为 8-11，需要减 8 为真实物理通道号）。以 MR527 为例，dsp 配置 mailbox 通道 4，对应的则是写 DSP msgbox 通道 0 和读 ARM msgbox 通道 0，rv 配置 mailbox 通道 8，对应的是写 RV msgbox 的通道 0 和读 ARM msgbox 通道 0；如果同时支持 rv 和 dsp，ARM 这边的 msgbox 读通道应该注意要分开，不能同时让 RV 和 DSP 都使用通道 0；
  - mbox-names：mailbox 通道名字。

存储分配

- 第 1 行的 reserved-memory 表示该节点下分配的内存都是预留的内存，预留的内存区域一般是给特定的驱动程序使用的，它和 Linux 内核使用的内存区域不同，一般预留的内存功能和 Linux 内核的 DMA 或者 CMA 紧密相关。
- 如果某个节点的 compatible 属性为 shared-dma-pool，则表示该节点内存区域用作一组设备的 DMA 缓冲区共享池，此时，如果看到节点属性中有 no-map，则表示该内存不能被 Linux 内核映射为系统内存的一部分，需要从系统内存中分离出来，如果看到节点属性中有 reusable 属性，则表示该内存不用从系统内存分离出来，当特定驱动不使用这些内存的时候，OS 可以使用这些内存。注意的是，一个节点不能同时有 no-map 和 reusable 属性，因为它们是逻辑上的矛盾关系。
- 我们看到 reserved-memory 节点下的每个子节点都包含 no-map 属性，说明这些内存不能被 Linux 内核用作系统内存，实际上是留给 DSP 的系统使用的。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.1.3 defconfig 配置

Linux 端需要打开如下 defconfig（longan 仓库在根目录执行./build.sh menuconfig，tina5.0 仓库在根目录执行./build.sh menuconfig 或者 make kernel_menuconfig）：

```
AW_REMOTEPROC_HIFI4_BOOT //hifi4架构的boot驱动
AW_REMOTEPROC_E906_BOOT   //e906架构的boot驱动
AW_RPMSG_CTRL  //aw rpmsg ctrl端点，用于用户层创建ept节点
AW_REMOTEPROC //remoteproc驱动
RPMSG_VIRTIO //remoteproc通过virtio对接rpmsg层的核心层代码
```

打开上述几个配置项后，其他配置项会默认打开。

4.2 RTOS OpenAMP 软件框架

4.2.1 源码结构

```
rtos/lichee/rtos-components/thirdparty/openamp
    ├──── Kconfig
    ├──── Makefile
    ├──── open-amp
    │     ├──── rpmsg       /* RPMsg 相关驱动文件 */
    │     │     ├──── rpmsg.c
    │     │     └──── rpmsg_virtio.c
    │     ├──── virtio
    │     │     ├──── virtio.c     /* DSP内核使用的 virtio 驱动 */
    │     │     └──── virtqueue.c     /* 虚拟队列（virtqueue）相关的驱动文件，用于管理 Virtio 队列和 vring 队列。每个 RPMsg
          通道包含两个与之关联的虚拟队列：一个 tx virtq ueue 用于主处理器单向传输数据到远程处理器，另一个 rx virtqueue
          用于远程处理器单向传输数据到主处理器 */
    │     └──── remoteproc
    │           ├──── remoteproc.c     /* 和 Remoteproc 相关的驱动文件 */
    │           ├──── remoteproc_virtio.c
    │           └──── rsc_table_parser.c   /* DSP 的固件资源表配置相关的文件 */
    ├──── sunxi_helper
    │     └──── openamp.c       /* OpenAMP 相关应用文件，OpenAMP 的初始化就是由此文件中的 openamp_Init()函数完成
          的 */
    └──── virtual_driver
          └──── virt_uart.c     /* 虚拟串口相关驱动文件，双核之间是通过 RPMsg 来进行消息传递的，基于 RPMsg，OpenAMP
          向上又进行了一层封装，并抽象出了虚拟串口（可以理解为一个普通的串口），主处理器和远程处理器可通过虚拟串口
          来传输数据 */
```

4.2.2 defconfig 配置

RTOS 端需要打开如下 defconfig（dsp 在 rtos-dev/lichee/dsp 目录下执行./build.sh menuconfig，rv 在 rtos-dev 目录下执行 mrtos_menuconfig）：

```
COMPONENTS_OPENAMP
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 AMP 软件框架介绍

本章节主要介绍 AMP 框架。

5.1 AMP 各层之间的关联

5.1.1 层级结构

从系统的角度来看，AMP 框架如下图所示：

- Linux 端分为三个组件：RPMSG、VIRTIO、REMOTEPROC，各自作用如下：
  - RPMSG：异构通信驱动框架；
  - VIRTIO：异构通信实际的数据交互区域；
  - REMOTEPROC：异构小核加载、启动、停止功能，并且兼顾 VIRTIO 以及共享内存的初始化部分。
- 通信方式使用中断 + 共享内存的方式进行通信；
- RTOS 端使用 OPENAMP 框架，其实现部分基本与 Linux 端的 RPMSG、VIRTIO、REMOTEPROC 类似，但由于没有用户态与内核态、物理地址与虚拟地址的限制，故实现方式较 Linux 端稍简单。

5.1.2 层级作用

5.1.2.1 virtio(虚拟化模块)

Virtio 是一个提供共享内存管理的虚拟设备框架，Virtio 中的 vring 是指向数据缓冲区指针的 FIFO 队列，有两个单向的 vring：

- 一个 vring 专用于发送到远程处理器的消息；
- 另一个 vring 用于从远程处理器接收的消息。

两个 vring 组成一个环形，Linux 和 DSP 的数据通过 vring 的缓冲区来共享，vring 的缓冲区就是两个处理器的共享内存（Shared memory，也可称为 Vring buffer）。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.1.2.2 RPMsg(远程处理器消息传递)

RPMsg 框架位于 Virtio 的上层,RPMsg(Remote Processor Messaging）框架是一种基于 Virtio 的消息总线。

RPMsg 服务是通过 RPMsg 框架实现的。一共有两个 vring，分别用于发送和接收的消息，Vring buffers 缓冲区就是共享的内存。RPMsg 框架基于 Virtio 的 vrings，它通过 Virtio 的 vrings 向远程处理器发送消息，或者从远程处理器接收消息，当新消息已经在共享内存中时，mailbox 框架就会通知处理器已经有消息可以接收。

RPMsg 实际上是一种基于 Virtio 的消息总线，主要就是用于实现消息传递，可以认为 RPMsg 是一个与远程处理器通信的通道，每个通道都有一个本地源地址和远程目标地址，消息就可以在源地址和目标地址之间进行传输。

5.1.2.3 RemoteProc(远程处理器控制)

Remoteproc 框架是由 Texas Instrument 开发的，在此基础上 Mentor Graphics 公司开发了一种软件框架 OpenAMP，在这个框架下，主处理器上的 Linux 操作系统可以对远程处理器及其相关软件环境进行生命周期管理，即启动或关闭远程处理器。

（1）RemoteProc 框架的主要作用：

- CPUX 将 DSP 固件映像的代码段和数据段加载到 DSP 内存中，以便之后执行程序；
- 解析 DSP 固件资源表并设置关联的资源（固件中各个段的起始地址和大小等信息，Virtio 设备特性、vring 地址、大小和对齐信息）；
- 远程控制 DSP 内核固件的启动和停止；
- 为与 DSP 的通信建立 RPMsg 通信通道；
- 提供监视和调试远程服务（remoteprov 框架中已经使用 sysfs 和 debugfs 文件系统，可直接使用）。

（2）Remoteproc 控制过程

- 主处理器先启动，再引导协处理器启动。
- 主处理器通过 Remoteproc 框架先加载远程处理器的固件，然后再解析固件资源表并配置协处理器系统资源，创建 Virtio 设备。
- 主处理器加载并启动远端处理器，远端处理器开始运行。
- 远端处理器启动后，此时双核之间还不能进行核间通信，需要先创建 RPMsg 通道，并由主处理器发送第一条消息以后，双核之间才可以进行核间通信，才可以互发数据。
- 如果需要停止运行固件，主处理器也可以关闭固件，远端处理器停止运行。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2 remoteproc

本章节主要介绍 Linux 侧，remoteproc 驱动及其核心层相关软件框架。

5.2.1 resouce_table 资源表

（1）远端处理器的固件映像内容

远端处理器的固件映像一般包括以下内容：

- 资源表（resource_table）；
- 用户应用程序；
- RTOS 或者裸机相关代码；
- OpenAMP 库。

主处理器先将远端处理器的固件加载到远程处理器的内核中，解码固件映像并获取关联的资源，为固件代码段和数据留出内存，在启动远程处理器后，主处理器创建 RPMsg 通道用于远程通信。

（2）远端处理器需要的资源

远端处理器的需要的资源包含以下两种：

- 远端处理器在上电之前需要的系统资源，例如分配给远端处理器的连续物理内存，还有分配给远端处理器的外围设备（这些设备可以是保留的、未使用的）；
- 资源表（resource_table），其中包含资源条目，这些条目保存记录远端处理器支持的功能或存在的配置，如 Virtio 设备中的 vring 地址、vring 的大小等。

只有在满足所有资源的要求后，Remotecore 才会启动设备。

（3）resource_table 资源表

在内核源码 include/linux/remoteproc.h 文件，找到如下结构体 resource_table，它是固件资源表头：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
/**
 * struct resource_table - firmware resource table header
 * @ver: version number
 * @num: number of resource entries
 * @reserved: reserved (must be zero)
 * @offset: array of offsets pointing at the various resource entries
 *
 * A resource table is essentially a list of system resources required
 * by the remote processor. It may also include configuration entries.
 * If needed, the remote processor firmware should contain this table
 * as a dedicated ".resource_table" ELF section.
 *
 * Some resources entries are mere announcements, where the host is informed
 * of specific remoteproc configuration. Other entries require the host to
 * do something (e.g. allocate a system resource). Sometimes a negotiation
 * is expected, where the firmware requests a resource, and once allocated,
 * the host should provide back its details (e.g. address of an allocated
 * memory region).
 *
 * The header of the resource table, as expressed by this structure,
 * contains a version number (should we need to change this format in the
 * future), the number of available resource entries, and their offsets
 * in the table.
 *
 * Immediately following this header are the resource entries themselves,
 * each of which begins with a resource entry header (as described below).
 */
struct resource_table {
        u32 ver;                /* 版本号 */
        u32 num;                /* 可用的资源条目数量 */
        u32 reserved[2];        /* 保留（必须为零） */
        u32 offset[];           /* 指向各种资源条目的偏移量的数组 */
} __packed;
```

图 5-1: resource_table 结构体

紧随此固件资源表头的是资源条目本身，每个条目都以 “struct fw_rsc_hdr” 标头开头，条目本身的内容会紧跟在这个头之后，根据资源类型来解析。以下是资源条目标头开头：

```
/**
 * struct fw_rsc_hdr - firmware resource entry header
 * @type: resource type
 * @data: resource data
 *
 * Every resource entry begins with a 'struct fw_rsc_hdr' header providing
 * its @type. The content of the entry itself will immediately follow
 * this header, and it should be parsed according to the resource type.
 */
struct fw_rsc_hdr {     /* 固件资源入口头 */
        u32 type;       /* 资源类型 */
        u8 data[];      /* 资源数据 */
} __packed;
```

图 5-2: fw_rsc_hdr 结构体

一些资源条目仅仅是公告，如其中主机被告知特定的 Remoteproc 配置，其它条目需要主机做一些事情（例如分配系统资源）。有时需要协商固件请求资源，一旦分配资源，主机应提供其详

版权所有 © 珠海全志科技股份有限公司。保留一切权利

细信息（如分配的内存区域的地址）。以下是当前支持的各种资源类型：

```
/**
 * enum fw_resource_type - types of resource entries
 *
 * @RSC_CARVEOUT:   request for allocation of a physically contiguous
 *                  memory region.
 * @RSC_DEVMEM:     request to iommu_map a memory-based peripheral.
 * @RSC_TRACE:      announces the availability of a trace buffer into which
 *                  the remote processor will be writing logs.
 * @RSC_VDEV:       declare support for a virtio device, and serve as its
 *                  virtio header.
 * @RSC_LAST:       just keep this one at the end of standard resources
 * @RSC_VENDOR_START:   start of the vendor specific resource types range
 * @RSC_VENDOR_END:     end of the vendor specific resource types range
 *
 * For more details regarding a specific resource type, please see its
 * dedicated structure below.
 *
 * Please note that these values are used as indices to the rproc_handle_rsc
 * lookup table, so please keep them sane. Moreover, @RSC_LAST is used to
 * check the validity of an index before the lookup table is accessed, so
 * please update it as needed.
 */
enum fw_resource_type {                         /* 资源条目的类型 */
        RSC_CARVEOUT            = 0,            /* 请求分配物理上连续的内存区域 */
        RSC_DEVMEM              = 1,            /* 请求 iommu_map 一个基于内存的外设 */
        RSC_TRACE               = 2,            /* 宣布跟踪缓冲区的可用性，远程处理器将在其中写入日志 */
        RSC_VDEV                = 3,            /* 声明对 Virtio 设备的支持，并作为其 Virtio 标头。 */
        RSC_LAST                = 4,            /* 把这个放在标准资源的末尾 */
        RSC_VENDOR_START        = 128,          /* 供应商特定资源类型范围的开始 */
        RSC_VENDOR_END          = 512,          /* 供应商特定资源类型范围的末尾 */
};
```

图 5-3: fw_resource_type 资源条目的类型

以上这些值用作 rproc_handle_rsc() 函数（在 remoteproc_internal.h 文件中）查找表的索引。

rtos 端的 resource table 定义在 rtos/lichee/rtos-components/thirdparty/openamp/include/openamp/remoteproc.h

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
/**
 * enum fw_resource_type - types of resource entries
 *
 * @RSC_CARVEOUT:   request for allocation of a physically contiguous
 *                  memory region.
 * @RSC_DEVMEM:     request to iommu_map a memory-based peripheral.
 * @RSC_TRACE:      announces the availability of a trace buffer into which
 *                  the remote remoteproc will be writing logs.
 * @RSC_VDEV:       declare support for a virtio device, and serve as its
 *                  virtio header.
 * @RSC_VENDOR_START: start of the vendor specific resource types range
 * @RSC_VENDOR_END  : end of the vendor specific resource types range
 * @RSC_LAST:       just keep this one at the end
 *
 * For more details regarding a specific resource type, please see its
 * dedicated structure below.
 *
 * Please note that these values are used as indices to the rproc_handle_rsc
 * lookup table, so please keep them sane. Moreover, @RSC_LAST is used to
 * check the validity of an index before the lookup table is accessed, so
 * please update it as needed.
 */
enum fw_resource_type {
    RSC_CARVEOUT = 0,
    RSC_DEVMEM = 1,
    RSC_TRACE = 2,
    RSC_VDEV = 3,
    RSC_LAST = 4,
    RSC_VENDOR_START = 128,
    RSC_AW_TRACE = 129,
    RSC_AMP_USER_RESOURCE = 130,
    RSC_AW_AW_MISC_RSC=131,
    RSC_VENDOR_END = 512,
};
```

图 5-4: rtos_resource_table

5.2.2 Linux 下 remoteproc 相关 API

5.2.2.1 rproc 结构体

(1) rproc 结构体

rprroc 结构体和远程处理器相关，在include/linux/remoteproc.h文件中定义，具体内容如下所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
struct rproc {
    struct list_head node;              /* rproc 对象的节点 */
    struct iommu_domain *domain;        /* iommu 域 */
    const char *name;                   /* rproc 的可读名称，即远程处理器的名称 */
    const char *firmware;               /* 要加载的固件文件名字 */
    void *priv;                         /* 芯片厂商保存自己私有数据的指针 */
    struct rproc_ops *ops;              /* rproc_ops 结构体是芯片厂商做好的，用于加载/启动/停止固件 */
    struct device dev;                  /* 用于引用计数和常见 Remoteproc 行为的虚拟设备 */
    atomic_t power;                     /* 需要此 rproc 启动的用户的引用计数 */
    unsigned int state;                 /* 设备状态 */
    enum rproc_dump_mechanism dump_conf;
    struct mutex lock;                  /* 保护 rproc 并发操作的锁 */
    struct dentry *dbg_dir;             /* 这个 rproc 设备的 debugfs 目录 */
    struct list_head traces;            /* 跟踪缓冲区列表 */
    int num_traces;                     /* 跟踪缓冲区的数量 */
    struct list_head carveouts;         /* 物理连续内存分配列表 */
    struct list_head mappings;          /* 我们启动的 iommu 映射列表，关闭时需要 */
    u64 bootaddr;                       /* 加载的首地址 */
    struct list_head rvdevs;            /* 远程 Virtio 设备列表 */
    struct list_head subdevs;           /* 子设备列表，跟踪运行状态 */
    struct idr notifyids;               /* idr 用于动态分配 rproc 范围内的唯一通知 ID */
    int index;                          /* 这个 rproc 设备的索引 */
    struct work_struct crash_handler;   /* 处理崩溃的工作队列 */
    unsigned int crash_cnt;             /* 崩溃计数器 */
    bool recovery_disabled;             /* 如果恢复被禁用，则标记该状态 */
    int max_notifyid;                   /* 最大分配的通知 ID */
    struct resource_table *table_ptr;   /* 指向有效资源表的指针 */
    struct resource_table *clean_table;
    struct resource_table *cached_table;  /* 资源表的副本 */
    size_t table_sz;                    /* cached_table 的大小 */
    bool has_iommu;                     /* 指示远程处理器是否在 MMU 后面的标志 */
    bool auto_boot;                     /* 指示是否应自动启动远程处理器的标志 */
    struct list_head dump_segments;     /* 固件中的段列表 */
    int nb_vdev;                        /* 当前由 rproc 处理的 Virtio 设备数量 */
    u8 elf_class;
    u16 elf_machine;
    struct cdev cdev;
    bool cdev_put_on_release;
};
```

rproc 结构体在 remoteproc 框架中的一个重要结构体，代表物理远程处理器设备。

(2) sunxi_rproc 结构体

在 rproc 结构体中，存在一个成员变量 **void *priv**，它是芯片厂商保存自己私有数据的指针，我司的私有数据结构体为 sunxi_rproc 结构体，具体内容如下：

```
struct sunxi_rproc {
    struct sunxi_rproc_priv *rproc_priv;  /* dsp/riscv private resources */
    struct sunxi_rproc_standby *rproc_standby;

    struct sunxi_mbox mb;
    struct workqueue_struct *workqueue;
    struct list_head list;
    struct rproc *rproc;

    struct device_node *np;
    /* whether using the firmware which is provided by kernel remoteproc framework(request_firmware) */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    int is_using_kernel_fw;
    /* private firmware */
    const struct firmware *fw;
    /* members below is used to save resource table info when we release private firmware */
    uint32_t fw_rsc_table_size;
    uint8_t fw_rsc_table_data[RPROC_MAX_RSC_TABLE_SIZE];

#ifdef AW_RPROC_SUPPORT_ZERO_COPY_MEM_FW
    int is_zero_copy_mem_fw_active;
    const char *zero_copy_mem_fw_name;
#endif

    void __iomem *rsc_table_va;
    bool is_booted;
    /* only boot on remote core without rsc table */
    bool only_boot;
    bool skip_shutdown;
    char *name;
#if IS_ENABLED(CONFIG_AW_RPROC_SUBDEV)
    struct sunxi_rproc_subdev subdev;
#endif
    struct fw_rsc_misc_rsc_data *misc_rsc_data;

    int is_nmi_available;
    uint32_t wait_nmi_complete_timeout;

#ifdef CONFIG_AW_RPROC_TRACE_EVENT_PARSER
    aw_trace_event_parser_t trace_event_parser;
#endif
};
```

(3) sunxi_rproc_priv 结构体

在 sunxi_rproc 结构体中含有 struct sunxi_rproc_priv *rproc_priv，它是指向 DSP/RV 私有资源的指针，而 sunxi_rproc_priv 结构体中含有每个异构小核的私有资源，sunxi_rproc_priv 结构体具体内容如下所示：

```
struct sunxi_rproc_priv {
    const char *name;
#if IS_ENABLED(CONFIG_SUNXI_RPROC_SHARE_IRQ)
    const char *share_irq;
#endif
    struct rproc *rproc;
    struct sunxi_rproc_memory_mapping *mem_maps;
    struct sunxi_rproc_ops *ops;
    struct device *dev;
    struct list_head list;
    int mem_maps_cnt;
    int irq;
    u32 pc_entry;
    void __iomem *io_base;
    void *rproc_cfg;
    void *standby_priv;
    void *priv;
    struct device_node *np;
    bool auto_boot;
};
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2.2.2 启动远程处理器

在remoteproc_sysfs.c文件中创建 coredump、recovery、firmware、state、name 节点，当向 state 节点写入 start 时，在 state_store 函数中调用 rproc_boot 函数，实现启动远程处理器。

rproc_boot() 函数用于启动远程处理器，其定义如下：

```
int rproc_boot(struct rproc *rproc)
{
    const struct firmware *firmware_p;
    struct device *dev;
    int ret;

    if (!rproc) {
        pr_err("invalid rproc handle\n");
        return -EINVAL;
    }

    dev = &rproc->dev;
    /* 获取互斥锁 */
    ret = mutex_lock_interruptible(&rproc->lock);
    if (ret) {
        dev_err(dev, "can't lock rproc %s: %d\n", rproc->name, ret);
        return ret;
    }
    /* 如果远程处理器的状态是RPROC_DELETED，则解锁 */
    if (rproc->state == RPROC_DELETED) {
        ret = -ENODEV;
        dev_err(dev, "can't boot deleted rproc %s\n", rproc->name);
        goto unlock_mutex;
    }

    /* 如果rproc已经通电，则跳过引导过程 */
    if (atomic_inc_return(&rproc->power) > 1) {
        ret = 0;
        goto unlock_mutex;
    }

    if (rproc->state == RPROC_DETACHED) {
        /* 如果远程处理器的状态是RPROC_DETACHED，则执行rproc_attach */
        dev_info(dev, "attaching to %s\n", rproc->name);
        ret = rproc_attach(rproc);
    } else {
        dev_info(dev, "powering up %s\n", rproc->name);

        /* 加载固件到内存 */
        ret = request_firmware(&firmware_p, rproc->firmware, dev);
        if (ret < 0) {
            dev_err(dev, "request_firmware failed: %d\n", ret);
            goto downref_rproc;
        }
        /* 获取固件并用固件启动远程处理器 */
        ret = rproc_fw_boot(rproc, firmware_p);
        /* 释放固件 */
        release_firmware(firmware_p);
    }

downref_rproc:
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    if (ret)
        atomic_dec(&rproc->power);
unlock_mutex:
    mutex_unlock(&rproc->lock);
    return ret;
}
```

- **mutex_lock_interruptible()**：是为了防止多个线程同时访问共享数据，也就是在操作设备时，通过上锁的方式来阻塞其他线程等待。
- **rproc_fw_boot()**：在函数中会先通过 rproc_fw_sanity_check() 函数检查固件是否合法，然后通过 rproc_enable_iommu() 函数来使能 iommu，通过 rproc_get_boot_addr() 函数从固件中获取 boot 启动地址，rproc_parse_fw() 函数从固件加载资源表、核心转储段列表等信息，最后通过调用 rproc_start() 函数来实现启动远程处理器。

对于 rproc_start 函数的具体调用关系如下：

```
->rproc_start()
  ->rproc_load_segment()//将elf固件load到memory中，调用的回调是我们sunxi_rproc.c里实现的
      sunxi_rproc_elf_load_segments，这里的将dsp固件按段memcpy到物理地址对应的虚拟地址上
  ->rproc_find_loaded_rsc_table()//将resource_table拷贝到rproc->cached_table中
  ->rproc_prepare_subdevices()
  ->rproc->ops->start()//sunxi_rproc.c里实现的对dsp的具体的启动方法
  ->rproc_subdevice_start()//这里调用的其实是rproc_vdev_do_start，这个函数是在一开始的rproc_handle_vdev注册的回
      调函数,rproc_handle_vdev函数里会实现对vdevbuffer划分成vring的相关操作
    ->rproc_vdev_do_start()
      ->rproc_add_virtio_dev()//添加virtio device，申请coherent内存，并且最后注册virtio device（重点）
```

5.2.2.3 停止远程处理器

前面使用 rproc_boot() 启动远程处理器，这里 rproc_shutdown() 函数用于关闭远程处理器。同样的，当向 **state** 节点写入 **stop** 时，在 state_store() 函数中调用 rproc_shutdown() 函数，实现停止远程处理器。rproc_shutdown() 函数定义如下：

```
void rproc_shutdown(struct rproc *rproc)
{
    struct device *dev = &rproc->dev;
    int ret;
    /* 获取锁 */
    ret = mutex_lock_interruptible(&rproc->lock);
    if (ret) {
        dev_err(dev, "can't lock rproc %s: %d\n", rproc->name, ret);
        return;
    }

    /* 如果仍然需要远程过程，退出 */
    if (!atomic_dec_and_test(&rproc->power))
        goto out;
    /* Virtio 设备被 rproc_stop() 销毁 */
    ret = rproc_stop(rproc, false);
    if (ret) {
        atomic_inc(&rproc->power);
        goto out;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    }
    /* 清理所有获得的资源 */
    rproc_resource_cleanup(rproc);
    /* 如果需要的话释放硬件资源 */
    rproc_unprepare_device(rproc);

    rproc_disable_iommu(rproc);

    /* Free the copy of the resource table */
    kfree(rproc->cached_table);
    rproc->cached_table = NULL;
    rproc->table_ptr = NULL;
out:
    mutex_unlock(&rproc->lock);
}
```

对于远程处理器的关闭，其过程与启动远程处理器的过程相反。首先调用 rproc_stop() 函数来停止远程处理器的任何子设备，rproc_stop() 函数分析过程如下:

```
->rproc_stop()
  ->rproc_stop_subdevices()//关闭子设备
  ->rproc_reset_rsc_table_on_stop()//无法再访问已安装的资源表
  ->rproc->ops->stop()//sunxi_rproc.c里实现的对dsp的具体的停止方法
  ->rproc_unprepare_subdevices()
```

最后通过调用 rproc_resource_cleanup() 函数清理所有获得的资源，清理调试跟踪条目，清除 carveout_memories 文件等。

5.2.3 msgbox 通道的申请与释放

在模块驱动的 sunxi_rproc_probe() 函数中，进行对 msgbox 通道的申请，具体的申请函数调用如下所示：

```
->sunxi_rproc_probe()
  ->sunxi_rproc_request_mbox()//初始化mailbox结构体
    ->mbox_request_channel_byname()//根据名字请求通道
```

在模块驱动的 sunxi_rproc_remove() 函数中，进行对 msgbox 通道的释放，具体的释放函数调用如下所示：

```
->sunxi_rproc_remove()
  ->sunxi_rproc_free_mbox()
    ->mbox_free_channel()//根据名字释放通道
```

5.2.4 Linux 侧 sysfs 和 debugfs 各个节点的作用

在 remoteproc 框架中的remoteproc_debugfs.c和remoteproc_sysfs.c中分别创建了 debugfs 和 sysfs 节点。

(1) sysfs 节点

版权所有 © 珠海全志科技股份有限公司。保留一切权利

在 remoteproc 框架中向用户提供了 **coredump**、**recovery**、**firmware**、**state**、**name** 节点，其路径如下：

```
/sys/kernel/class/remoteproc/remoteproc0/state
/sys/kernel/class/remoteproc/remoteproc0/name
/sys/kernel/class/remoteproc/remoteproc0/firmware
/sys/kernel/class/remoteproc/remoteproc0/recovery
/sys/kernel/class/remoteproc/remoteproc0/coredump
```

其作用分别为：

- **state**：通过输入不同的命令来控制远程处理器的状态；
- **name**：显示远程处理器的名称；
- **firmware**：通过输入来修改固件名称；
- **recovery**：recovery 的内容可以是 enabled、disabled 或 recovery 三者之一，用于控制恢复机制的行为。
  - enabled 表示远程处理器将在崩溃时可以自动恢复；
  - disabled 表示远程处理器在崩溃时将保持崩溃状态；
  - recovery 表示如果远程处理器处于崩溃状态，此功能将触发立即恢复，而不用手动更改或检查恢复状态（启用/禁用）。
- **coredump**：coredump 的内容可以是 inline、disabled 或 default 三者之一，用于控制核心转储机制的行为。
  - inline 表示 coredump 不会被复制到单独的缓冲区，恢复过程必须等到数据被用户空间读取；
  - disabled 表示这是默认的核心转储机制。恢复将在不收集任何转储的情况下继续进行；
  - default 表示当 remoteproc 崩溃时，整个核心转储将被复制到一个单独的缓冲区并暴露给用户空间。

(2) debugfs 节点

在 remoteproc 框架中向用户提供了 **name**、**recovery**、**crash**、**resource_table**、**carveout_memories**、**coredump** 节点，其路径如下：

```
/sys/kernel/debug/remoteproc/remoteproc0/name
/sys/kernel/debug/remoteproc/remoteproc0/recovery
/sys/kernel/debug/remoteproc/remoteproc0/crash
/sys/kernel/debug/remoteproc/remoteproc0/resource_table
/sys/kernel/debug/remoteproc/remoteproc0/carveout_memories
/sys/kernel/debug/remoteproc/remoteproc0/coredump
```

其作用分别为：

- **name**：显示远程处理器的名称；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- **recovery**：recovery 的内容可以是 enabled、disabled 或 recovery 三者之一，用于控制恢复机制的行为。
  - enabled 表示远程处理器将在崩溃时可以自动恢复；
  - disabled 表示远程处理器在崩溃时将保持崩溃状态；
  - recovery 表示如果远程处理器处于崩溃状态，此功能将触发立即恢复，而不用手动更改或检查恢复状态（启用/禁用）。
- **crash**：记录系统崩溃时候的有关信息；
- **resource_table**：记录了 DSP 的资源信息（资源表）；
- **carveout_memories**：记录 DSP 的内存分配情况；
- **coredump**：coredump 的内容可以是 inline、disabled 或 default 三者之一，用于控制核心转储机制的行为。
  - inline 表示 coredump 不会被复制到单独的缓冲区，恢复过程必须等到数据被用户空间读取；
  - disabled 表示这是默认的核心转储机制。恢复将在不收集任何转储的情况下继续进行；
  - default 表示当 remoteproc 崩溃时，整个核心转储将被复制到一个单独的缓冲区并暴露给用户空间。

5.3 RPmsg

5.3.1 相关结构体

(1) rpmsg_device 结构体

rpmsg_device 结构体是属于 rpmsg 总线的设备，其具体内容如下：

```
struct rpmsg_device {
    struct device dev;
    struct rpmsg_device_id id;
    char *driver_override;
    u32 src;
    u32 dst;
    struct rpmsg_endpoint *ept;
    bool announce;
    bool little_endian;
    const struct rpmsg_device_ops *ops;
};
```

其成员变量为：

- **dev**: 设备结构；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- **id**：设备 id（用于匹配 rpmsg 驱动和设备）；
- **driver_override**：强制匹配的驱动名称；
- **src**: 本地地址；
- **dst**: 目的地址；
- **ept**：此通道的 rpmsg 端点；
- **announce**：如果设置，rpmsg 将宣布创建/删除此频道；
- **little_endian**：如果传输使用小端字节表示则为真。

(2) rpmsg_endpoint 结构体

rpmsg_endpoint 结构体将本地 rpmsg 地址绑定到它的使用设备上，其具体内容如下：

```
struct rpmsg_endpoint {
    struct rpmsg_device *rpdev;
    struct kref refcount;
    rpmsg_rx_cb_t cb;
    struct mutex cb_lock;
    rpmsg_rx_sig_t sig_cb;
    bool rx_done;
    u32 addr;
    void *priv;
    const struct rpmsg_endpoint_ops *ops;
};
```

其成员变量为：

- **rpdev**: rpmsg 通道设备；
- **refcount**：当它下降到零时，ept 被释放；
- **cb**: 接收回调处理程序；
- **cb_lock**：必须在访问/更改 cb 之前获取；
- **sig_cb**: rx 串行信号处理程序；
- **rx_done**：如果设置，rpmsg 端点支持 rpmsg_rx_done；
- **addr**: 本地 rpmsg 地址；
- **priv**：驱动程序使用的私有数据。

(3) rpmsg_hdr 结构体

struct rpmsg_hdr 结构体是所有 rpmsg 消息的通用标头，rpmsg 总线上发送或接收的每条消息都以此标头开头，其具体内容如下：

```
struct rpmsg_hdr {
    __rpmsg32 src;
    __rpmsg32 dst;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    __rpmsg32 reserved;
    __rpmsg16 len;
    __rpmsg16 flags;
    u8 data[];
} __packed;
```

其成员变量为：

- **src**: 源地址；
- **dst**: 目的地址；
- **reserved**：保留以供将来使用；
- **len**：数据长度（以字节为单位）；
- **flags**: 消息标志；
- **data**: len 字节的消息负载数据。

5.3.2 API 函数

5.3.2.1 rpmsg_send_offchannel_raw()

- 函数原型：static int rpmsg_send_offchannel_raw(struct rpmsg_device rpdev, u32 src, u32 dst, void data, int len, bool wait)；
- 功能：向远程处理器发送消息；
- 参数：
  - rpdev：rpmsg 通道；
  - src: 源地址；
  - dst: 目的地址；
  - data: 要发送的消息；
  - len: 消息长度；
  - wait：表示如果没有可用的 TX 缓冲区，调用者是否应该阻塞。
- 返回值：成功时返回 0，失败时返回错误码。

5.3.2.2 rpmsg_send()

- 函数原型：int rpmsg_send(struct rpmsg_endpoint ept, void data, int len)；
- 功能：将消息发送到远程处理器；
- 参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - ept：rpmsg 端点；
  - data：要发送的消息；
  - len：消息长度。
- 返回值：成功时返回 0，失败时返回错误码。

5.3.2.3 rpmsg_sendto()

- 函数原型：int rpmsg_sendto(struct rpmsg_endpoint ept, void data, int len, u32 dst)；
- 功能：将消息发送到远程处理器，指定目的地址；
- 参数：
  - ept：rpmsg 端点；
  - data：要发送的消息；
  - len：消息长度；
  - dst：目的地址。
- 返回值：成功时返回 0，失败时返回错误码。

5.3.2.4 rpmsg_send_offchannel()

- 函数原型：int rpmsg_send_offchannel(struct rpmsg_endpoint ept, u32 src, u32 dst, void data, int len)；
- 功能：使用明确的源/目标地址发送消息；
- 参数：
  - ept：rpmsg 端点；
  - src：源地址；
  - dst：目的地址；
  - data：要发送的消息；
  - len：消息长度。
- 返回值：成功时返回 0，失败时返回错误码。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.3.2.5 rpmsg_trysend()

- 函数原型：int rpmsg_trysend(struct rpmsg_endpoint ept, void data, int len)；
- 功能：向远程处理器发送消息；
- 参数：
  - ept：rpmsg 端点；
  - data：要发送的消息；
  - len：消息长度。
- 返回值：成功时返回 0，失败时返回错误码。

5.3.2.6 rpmsg_trysendto()

- 函数原型：int rpmsg_trysendto(struct rpmsg_endpoint ept, void data, int len, u32 dst)；
- 功能：将消息发送到远程处理器，指定目的地址；
- 参数：
  - ept：rpmsg 端点；
  - data：要发送的消息；
  - len：消息长度；
  - dst：目的地址。
- 返回值：成功时返回 0，失败时返回错误码。

5.3.2.7 rpmsg_trysend_offchannel()

- 函数原型：int rpmsg_trysend_offchannel(struct rpmsg_endpoint ept, u32 src, u32 dst, void data, int len)；
- 功能：使用明确的源/目标地址发送消息；
- 参数：
  - ept：rpmsg 端点；
  - src：源地址；
  - dst：目的地址；
  - data：要发送的消息；
  - len：消息长度。
- 返回值：成功时返回 0，失败时返回错误码。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 AMP 基本的使用方法

6.1 异构启停

在应用层通过向 state 节点传入命令的方式，控制远程处理器启停，以下以 DSP 的带起为例，其他小核的启停方式相同：

- 启动异构小核

```
echo amp_dsp.bin > /sys/class/remoteproc/remoteproc0/firmware
echo start > /sys/class/remoteproc/remoteproc0/state
```

其中，amp_dsp.bin 为 DSP 的镜像，已被编译进固件，若需替换，请将新的 DSP 固件推至小机端，重命名为 amp_dsp.bin，替换掉lib/firmware中的 amp_dsp.bin，并重新执行上述命令。

```
echo stop > /sys/class/remoteproc/remoteproc0/state
```

6.2 异构通信测试

以下示例可以通过 rpmsg 进行异构间通信测试：

- 只有在主处理器发送第一条消息以后，双核之间才可以进行核间通信，才可以互发数据。因此，当创建 debugfs 节点后，主处理器在 rpmsg_test_probe() 函数中调用 rpmsg_send() 函数主动向远端处理器发送第一条信息；
- 在 DSP 端调用 rpmsg_test_init() 接口，并创建 rpmsg 端点；
- 在主处理器的 debugfs 节点的 write 函数中先从用户层得到要传输的数据，并调用 rpmsg_send() 函数发送至 DSP；
- DSP 在接收到数据之后，调用 callback 函数，将接收到数据显示出来；
- 在 DSP 中调用 rpmsg_test_send() 接口（内部真实调用 rpmsg_send() 函数），发送数据至主处理器；
- 在主处理器接收到数据之后，callback 回调会调用 rpmsg_test_cb() 函数，将接收到的数据显示出来，完成 CPUX->DSP、DSP->CPUX 通信的完整回路，测试代码如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.2.1 Linux 端测试代码：

以下介绍 debugfs 节点的 write 函数以及 callback 回调会调用 rpmsg_test_cb() 函数，整体测试代码见附件 **Linux_rpmsg_test.c**。

```
static int rpmsg_test_cb(struct rpmsg_device *rpdev, void *data, int len,
                void *priv, u32 src)
{
  dev_info(&rpdev->dev, "incoming msg %s (src: 0x%x)\n",
    (char *)data, src);
  return 0;
}

static ssize_t rpmsg_test_send_write(struct file *filp,
            const char __user *userbuf,
            size_t count, loff_t *ppos)
{
  struct rpmsg_test *chip = filp->private_data;
  struct device *dev = &chip->rpdev->dev;
  int ret;

  if (count > RPMSG_TEST_MSG_LEN) {
    dev_err(dev, "Error: message length %zd greater than max allowed %d\n",
        count, RPMSG_TEST_MSG_LEN);
    return -EINVAL;
  }
  chip->msg = kzalloc(RPMSG_TEST_MSG_LEN, GFP_KERNEL);
  if (!chip->msg)
    return -ENOMEM;
  ret = copy_from_user(chip->msg, userbuf, count);
  if (ret) {
    ret = -EFAULT;
    goto out;
  }
  ret = rpmsg_send(chip->rpdev->ept, chip->msg, strlen(chip->msg));
  if (ret) {
    ret = -EFAULT;
    dev_err(dev, "Error: rpmsg send failed\n");
  }
out:
  kfree(chip->msg);
  return ret < 0 ? ret : count;
}
```

6.2.2 远端处理器测试代码：

以下介绍 DSP 端的 rpmsg_test_init()、rpmsg_test_send() 以及 DSP 在接收到数据之后，调用 callback 函数 rpmsg_ept_callback()，整体测试代码见附件 **RTOS_rpmsg_test.c**。

```
/* callback回调 */
static int rpmsg_ept_callback(struct rpmsg_endpoint *ept, void *data,
    size_t len, uint32_t src, void *priv)
{
  printf("%s recv %s\n", ept->name, (char *)data);
  return 0;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
}

static int rpmsg_test_init(void)
{
    void *thread;

    thread = kthread_create(cmd_rpmsg_test_thread, NULL, "rpmsg_test", 2048, 8);
    if(!thread)
        kthread_start(thread);
    else
        printf("fail to create rpmsg hearbeat thread\r\n");
    return 0;
}
FINSH_FUNCTION_EXPORT_CMD(rpmsg_test_init, rpmsg_test_init, init rpmsg test);

static int rpmsg_test_send(void)
{
    openamp_rpmsg_send(chip->ept, MSG, strlen(MSG));
    return 0;
}
FINSH_FUNCTION_EXPORT_CMD(rpmsg_test_send, rpmsg_test_send, rpmsg test send);
```

6.2.3 驱动层 rpmsg 传输

以 bsp 仓库 rpmsg_heartbeat.c 为例

```
static struct rpmsg_device_id rpmsg_driver_hearbeat_id_table[] = {
    { .name = "sunxi,rpmsg_heartbeat_arm" },
    { .name = "sunxi,rpmsg_heartbeat" },
    { },
};

static struct rpmsg_driver rpmsg_sample_client = {
    .drv = {
        .name   = KBUILD_MODNAME,
#ifdef CONFIG_PM_SLEEP
        .pm = &sunxi_rpmsg_hearbeat_pm_ops,
#endif
    },
    .id_table   = rpmsg_driver_hearbeat_id_table,
    .probe      = rpmsg_heartbeat_probe,
    .callback   = rpmsg_heartbeat_cb,
    .remove     = rpmsg_heartbeat_remove,
};
module_rpmsg_driver(rpmsg_sample_client);
```

需要声明并注册 rpmsg_driver

- rpmsg_driver_hearbeat_id_table：rpmsg driver 在 rpmsg bus 是通过 name match 的，所以要求异构核的 openamp_open_ept 的端点名和内核 rpmsg driver 的 id_table 匹配。

当对端一般为异构 rtos 系统，以 rpmsg heartbeat 为例：rtos/lichee/rtos-components/thirdparty/openamp/rpmsg_demo/hearbeat.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
heart->ept = openamp_ept_open(heart->name, 0, heart->src, heart->dst,
        heart, rpmsg_ept_callback, rpmsg_unbind_callback);
//rpmsg_ept_callback为接收回调，rpmsg_unbind_callback为端点unbind回调
```

调用 openamp_ept_open，则 client 端会通过 rpmsg_ns ept 端点把端点名发给内核，rpmsg_bus 监听到后会遍历已注册的 rpmsg_driver 触发名字相同的 driver 的 probe 函数，本例子中为 rpmsg_heartbeat_probe 函数。

- rpmsg_heartbeat_cb: 为端点接收的回调函数，可解析接收到的 rpmsg 报文。
- rpmsg_send: 内核可以使用 rpmsg_send(ept, buf, len) 向异构核发送报文。
- rpmsg_heartbeat_remove: 当对端异构核关闭端点时，驱动会触发该回调。

rpmsg_driver 通过 register_rpmsg_driver 接口注册到总线上，或使用内核驱动注册函数 module_rpmsg_driver。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。